# AI Tender Qualification Assistant

Technical Bid Analysis for refinery and industrial tenders

RHQ AI/ML Hackathon 2026 • Category 05: Workflow Automation & Others (document intelligence) • Working prototype, live

An evaluation engineer opens a tender, uploads the NIT and each bidder's credentials, and gets the committee's 18-column Technical Bid Analysis back, with every figure traceable to the page it came from. The app reads each tender's own NIT and judges the bidders against that NIT's clauses, not against rules written into the software. Every decision stays a suggestion until an engineer confirms it. **It is running today** and has been used on real IOCL tenders, including an 896-page bidder dossier read in 2 minutes 8 seconds.

## Gate 1 — The problem

Every works tender ends in a Technical Bid Analysis. For each bidder, each submitted work order is checked against the NIT's pre-qualification criteria, and the committee records accepted, rejected or query, with reasons. Today that is done by reading PDFs and typing a spreadsheet.

- **The evidence is bulky.** One bidder in our live test submitted an **896-page, 17 MB** dossier holding 29 work orders, completion certificates, tax statements and a chartered accountant's certificate. Most pages are scans, so text search does not work on them.
- **The checks are fiddly.** Similar nature of work, ordered value against executed value, GST inclusive or not, annualisation for rate contracts, sub-contracts and the owner's approval, the experience window, and the 1 / 2 / 3-works value options — each judged per work order and then across the bidder.
- **Every tender differs.** The values, the similar-nature definition, the window and any relaxation come from that tender's NIT, so a tool with rules baked in is wrong from the second tender onwards.
- **It must stand scrutiny months later**, against the page of the document each number came from.
- **Effort:** about **2–3 working days** of an engineer's time for a tender with 4–6 bidders of this size, and the risk that one missed clause changes who is qualified.
- **And it repeats constantly.** Paradip Refinery alone floats about **300–400 service and works tenders every year** — purchase tenders, which use a different TBA format, are over and above this. Every one of them ends in a hand-built sheet.

## Gate 2 — The solution and the prototype

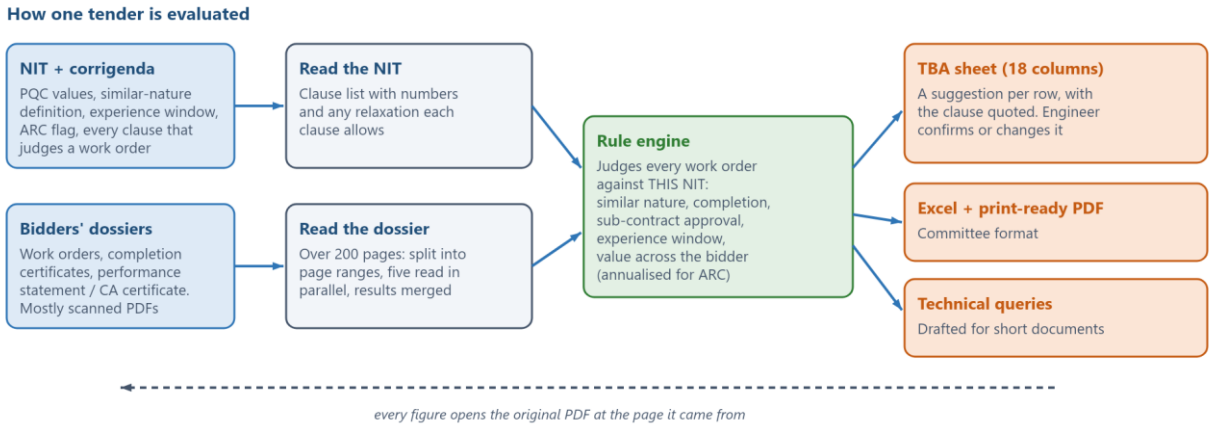



*Figure 1 — The NIT sets the rules; the dossiers are read against them; the engineer confirms; the committee's sheet, the exports and the queries follow.*

### What it does

1. **Reads the NIT** — PQC values, the similar-nature definition with its clause number, the experience window, the ARC flag, and every clause that judges a work order, with any relaxation it allows. Corrigenda are uploaded on top and replace only what they change.
2. **Reads each bidder's dossier** — work orders, completion certificates, dates, values and GST basis, sub-contract signs, JV shares, composite orders, and the bidder's own performance statement or CA certificate.

3. **Judges each work order against that NIT** — similar nature, completion proof, sub-contract approval, experience window, and the value criterion judged across the bidder (one work above the 1-work value, or two or three works each above theirs, annualised for ARC tenders).
4. **Writes the committee's sheet** — the 18-column TBA with a remark for every decision, exported to Excel and to a print-ready PDF in the committee's format.
5. **Backs every figure with its page** — the original PDF is kept; clicking a value opens it at that page and highlights the text.
6. **Drafts the technical queries** for bidders whose documents fall short.

### The idea that makes it work: the NIT decides

- **Nothing is hard-coded.** Values, window, similar-nature definition and annualisation come from the tender's own NIT. Where the NIT gives no figure, the app says the criterion was not checked rather than inventing one.
- **Every rule quotes its clause:** "Annualised value Rs. 90.62 Lacs meets Rs. 79.28 Lacs only for 2 similar works each … (Clause No. 11.2 of NIT)".
- **A relaxation is put to the engineer.** If a clause allows one — a lower value for MSE bidders, say — the remark carries it and asks for a decision instead of applying the standard rule.
- **Silence is declared.** Where a NIT says nothing on a point, the remark says the standard rule was used and asks for confirmation.
- **Clause numbering is never assumed.** Rules match on the subject of a clause, so a NIT that numbers its value criterion 10.2 instead of 11.2 works unchanged.

### Prototype status

Live and usable today at **tender-scrutiny-assistant.web.app**. "Try the demo without an account" opens a sample tender with 16 bidders already evaluated. Real tenders (9070C26C12, 9070C26C15 and others) have been run end to end: NIT read, dossiers extracted, sheet confirmed, Excel and PDF exported.

## Gate 3 — The benefit

| Per tender (4–6 bidders) | Today | With the app |
|---|---|---|
| Reading dossiers and extracting work orders | 1–2 days | About 10 minutes of machine time (unattended) |
| Applying the PQC checks and writing remarks | ½–1 day | Immediate; the engineer reviews and confirms |
| Typing and formatting the committee sheet | 2–4 hours | Exported, already in the committee's format |
| **Engineer time in total** | **16–24 hours** | **About 2–3 hours of review** |

**Tangible: about 14–21 engineer-hours saved per tender, roughly 85% of the effort.** Paradip Refinery alone evaluates **300–400 service and works tenders a year**, so the saving at one refinery is of the order of **4,200–8,400 engineer-hours — about 525–1,050 working days, or two to five engineers' full year** returned to engineering work. Purchase tenders are not counted here, and across the other refineries and divisions the same rate runs into tens of thousands of hours. (Hours per tender are from our own evaluation practice; the tender count is Paradip's actual annual volume.)

- **Fewer missed clauses.** In the live test the app flagged five orders the bidder claimed but never documented — including an IOCL GeM contract — which is exactly the kind of gap that returns as a query or a complaint later.
- **A defensible file.** Every figure links to its page in the original PDF, and every decision carries the clause it rests on, months after the award.
- **Consistency between engineers and refineries.** The same NIT produces the same reading of the rules, whoever runs it.
- **Faster queries, shorter cycle.** Technical queries are drafted from what is actually missing, so the round trip with bidders starts the same day.
- **Knowledge retained.** The reasoning is in the sheet, not in one experienced engineer's head.

## Gate 4 — Sustenance and scalability

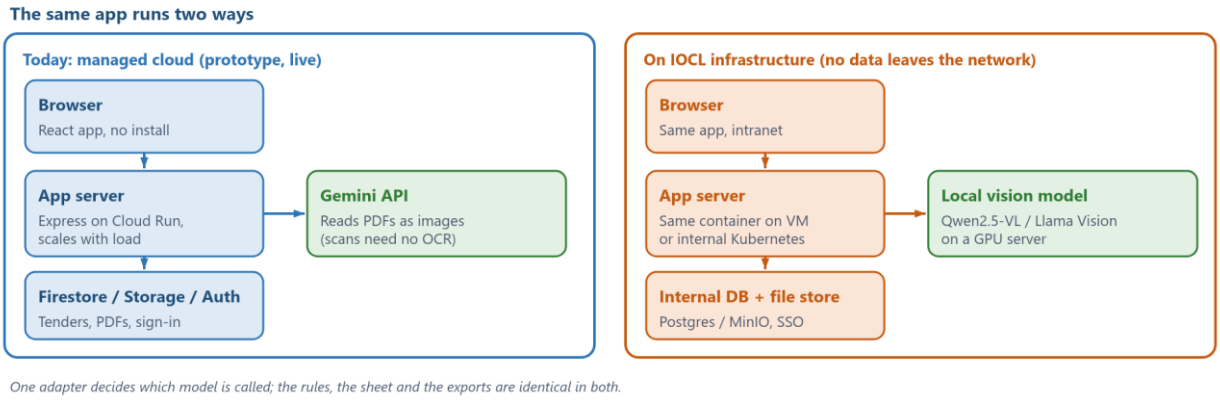



*Figure 2 — The prototype runs on managed cloud today; the same container runs inside IOCL with a local model and internal storage.*

## Can IOCL run it on its own servers, with local models?

Yes, and the design was kept that way from the start. The app is a container plus one adapter that calls the AI; the rules, the sheet and the exports do not know which model answered.

- **Hosting:** the same image runs on an internal VM or Kubernetes behind SSO; no internet dependency for the app itself.
- **Data:** tenders and PDFs move to an internal database and object store (Postgres / MinIO, or the existing enterprise equivalents). No bid document leaves the network.
- **Model:** the adapter is swapped to an internal endpoint. A **vision-capable** model is needed because bid documents are scans — Qwen2.5-VL 32B / 72B or Llama 3.2 Vision on a GPU server (roughly one A100/L40S class GPU for a refinery's load), served through vLLM with an OpenAI-compatible API. Where only a text model is available, an OCR stage (PaddleOCR / Tesseract) goes in front of it.
- **Cost:** on the managed path the AI cost is a few rupees per dossier; on-premise it is the GPU server, amortised, with no per-page charge.
- **Accuracy check when switching:** the same dossiers can be re-run and compared against the sheet already confirmed, so a model change is verified against real tenders rather than trusted.

## Scale

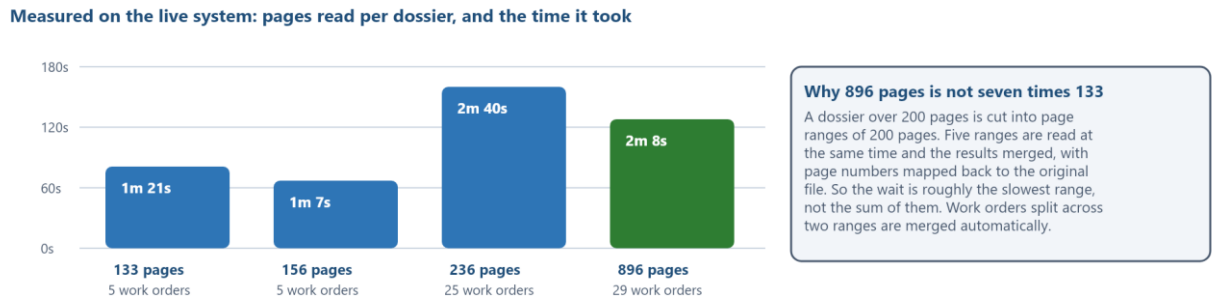



*Figure 3 — Measured on the live system. A dossier over 200 pages is read as five page ranges in parallel, so 896 pages take about as long as the slowest range.*

- **Per tender:** 4–10 bidders, dossiers of 100–900 pages, read in minutes, not hours.
- **Concurrent users:** the server scales out per request (Cloud Run today, replicas on-premise); tenders are independent, so more evaluators simply mean more instances.
- **A refinery's whole year:** 400 tenders at 4–6 dossiers each is about 2,000 dossiers, roughly 8 a working day. At the measured rates one instance absorbs that with room to spare, and instances are added per request, not per refinery.
- **More refineries:** nothing in the code is tied to a unit or a tender type. A new NIT is a new upload, not a new release.
- **Beyond works tenders:** the same reading and rule engine fits supply and service tenders, and any check where a document must be judged against a written clause.

## Keeping it alive

- **One rules module.** Every decision lives in a single TypeScript file used by the sheet, the edit window and both exports, so they cannot disagree. Rule changes are covered by automated checks built from real tenders.
- **No per-tender configuration** to maintain: the NIT is the configuration.
- **Model independence.** Five models are configured with automatic fallback; the adapter means a new model, or an internal one, is a configuration change.
- **Known limits, already scheduled:** automatic splitting of PDFs over 20 MB, storing each bidder separately before large tenders reach the database's per-record ceiling, and evaluating turnover and other financial criteria that are read but not yet judged.
- **Handover:** the code is in a repository with the deployment commands, a backlog and design notes in plain language.

## Ease of implementation

1. **Nothing to install.** The evaluator opens a browser and signs in with the office Google account (or SSO on-premise).
2. **Upload the NIT.** The clauses, values and window are read in about half a minute; the engineer checks them and fills in the bid submission date if the NIT leaves it to GeM.
3. **Add bidders and upload their PDFs.** Each dossier is read in minutes, and duplicates across uploads are merged on confirmation.
4. **Review and confirm** the suggested rows, then export the Excel and PDF.

No integration with SAP or GeM is required, no training data has to be collected, and no model has to be trained: the NIT and the bid documents are the only inputs. A pilot needs one refinery, five past tenders whose outcome is already known, and two weeks — comparing the app's sheet against the committee's.

## How accurate is it?

We checked the 896-page dossier by hand: reading the 800 pages that carry a text layer, looking at every scanned page the app had not covered, and comparing against the bidder's own performance statement (31 orders) and a chartered accountant's certificate (27 orders).

| What was checked | Result |
|---|---|
| Work orders documented in the dossier | 28 |
| Found by the app | 27 (96%) |
| Rows invented by the app | 0 — every row is a real order in the file |
| Orders claimed by the bidder with no document in the file | 3 — correctly not created, and now flagged for a query |
| Work-order values correct (GST basis included) | 24 of 26 at the time of the check; the one GST error found is fixed |
| Completed values matching the CA certificate | 5 of 6 |

That check produced three improvements now in the app: the bidder's own list of claimed orders is read and compared with the extracted rows; a value counts as excluding GST only when the document says so in words; and a row with no client completion certificate carries what the CA certificate states, without treating it as the client's certificate.

## Where the engineer stays in charge

- Every row is a **suggestion until confirmed**; the export marks unconfirmed rows as pending.
- **A confirmed row is never changed** by the app. If a later edit puts it outside a NIT condition, the row gets a rule-check note and the engineer decides.
- **Nothing is invented.** A document with no work order in it adds no rows and says what the document is; a field the documents do not give stays blank with the reason in its place.
- **Duplicates are reported, not merged silently**, within an upload or across several.
- **Edit locks and named confirmations** keep a shared tender honest: who confirmed which row, and when.

## Try it

Live app: tender-scrutiny-assistant.web.app — “Try the demo without an account” opens a sample tender with 16 bidders evaluated and allows three PDF extractions a day. Signing in with Google gives the full app: your own tenders, shared with your team, with edit locks and confirmations recorded under your name.